EXCEPTION ONE

Autonomous recovery that accounts for everyone | Team project proposal | 31 October 2026

One-line pitch: When an event changes unexpectedly, Exception One finds the person whose needs the replacement plan overlooks, tests workable alternatives, and verifies an approved recovery.

01 Problem and opportunity

Last-minute venue changes, resource shortages, and scheduling disruptions can produce a solution that works for most attendees but fails someone with a hard constraint. Capacity may be sufficient while step-free access, required equipment, or a timing accommodation is not. Under time pressure, organisers may miss these linked requirements.

Target user: University and event operations teams managing multiple sessions, rooms, resources, and participant requirements. Initial scope: last-minute venue replacement for a university event. This is a plausible problem hypothesis; we have not established its prevalence through field research.

02 Product concept

Exception One is not a chatbot that suggests accessible venues. It is a constraint-aware operational recovery workflow that reads structured event state, traces dependencies, executes a recovery algorithm, applies approved changes to a simulated schedule, and verifies the outcome.

- Detect: An organiser marks a venue unavailable.
- Trace: Neo4j identifies affected sessions, participants, requirements, and resources.
- Check: Explicit rules flag which hard constraints are violated by the initial replacement.
- Recover: Daytona runs Python code to search feasible reassignments.
- Approve and apply: The organiser accepts a proposal; the backend updates the simulated schedule.
- Verify and record: The system checks all required constraints again and stores evidence in Neo4j.

03 Live demo: the overlooked attendee

Demo-only synthetic data: 200 participants, multiple sessions, three alternative venues, room equipment requirements, and one step-free-access requirement. The originally assigned room becomes unavailable. A replacement fits capacity but violates one attendee’s requirement.

BEFORE	AFTER VERIFIED RECOVERY
200 attendees have a nominal venue assignment, but one required accessibility constraint is violated.	A feasible alternative assignment is selected; explicit checks return zero unresolved hard constraints for the prepared scenario.

Live twist: Disable one of the alternative rooms and rerun. The recommendation must change or honestly report that no feasible solution exists. This shows a live solver, not a scripted answer.

04 Why this is more than ChatGPT with documents

A document chat can...	Exception One must...
Explain venue requirements and propose possible changes.	Read the current structured event and resource state.
Generate a plan from whatever data is uploaded.	Check every stated hard constraint against live operational records.
Suggest how to reschedule.	Execute a real recovery algorithm in Daytona.
Describe success.	Apply a proposed change only with approval, then rerun checks and record evidence.

Tool-enabled coding agents can also perform actions; our distinction is a specialised, repeatable workflow with explicit acceptance tests and auditability, not a claim that general-purpose AI is fundamentally incapable.

05 Sponsor integration: each has a real job

SPONSOR	ROLE IN THE MVP
Neo4j	Graph of Participant, Requirement, Session, Venue, Resource, and RecoveryAction; query affected dependencies and persist verified results.
Daytona	Isolated Python execution of the venue-assignment solver and post-change validation checks.
Nosana	AI inference to interpret a natural-language incident and explain the solver output; numerical feasibility comes from deterministic code.

06 MVP build plan

Must ship: Preloaded synthetic event; disruption trigger; Neo4j graph query; rule-based constraint detector; Daytona algorithm run; proposed reassignment; organiser approval in the demo interface; post-change verification; Neo4j outcome record.

Only if time allows: Nosana-powered natural-language input; interactive graph; second disruption; downloadable incident report. The team should confirm any sponsor-specific integration requirements with the organisers.

Out of scope: Real university systems, actual venue bookings, real attendee notifications, production accessibility certification, handling sensitive medical data, and unapproved external actions.

07 Proposed technical stack and ownership

COMPONENT	TECH / RESPONSIBLE WORK
Frontend	Vue 3: incident dashboard, violated-constraint alert, recovery approval, before/after view.
Backend	Python + FastAPI: orchestration, validation, event-state changes.
Graph	Neo4j AuraDB: schema, synthetic seed data, impact queries, run history.
Execution & AI	Daytona Python SDK: sandbox test/solver; Nosana API: incident interpretation and explanations.
Product/demo	Scope, demo script, verification proof, contingency demo.

08 Three-minute demonstration

TIME	DEMO BEAT
0:00–0:25	Problem: most attendees are served, but one exception is overlooked.
0:25–0:50	Trigger venue closure in the dashboard.
0:50–1:20	Show Neo4j-linked impact and the failed hard constraint.
1:20–1:55	Run the recovery algorithm in Daytona; show real execution status.
1:55–2:25	Review and approve feasible new room assignment.
2:25–3:00	Show verification and persisted recovery evidence; deliver closing line.

09 Risks and proof of success

- False confidence: Use explicit constraint data and verify checks; never let an LLM certify accessibility.
- Too much scope: Support one deterministic disruption with a small, credible dataset.
- Integration risk: First complete graph → execution → verified write-back; polish later.
- Novelty: Similar scheduling/accessibility solutions may exist; do not claim global originality without competitor research.
- Privacy: Use synthetic participant records for the hackathon.

Acceptance test: Changing or removing a venue causes the system to discover a constraint violation, execute a recovery in a genuine Daytona sandbox, update simulated operational state after approval, and independently verify/write back the result.

10 Decision required from the team

Agree on the venue-change scenario; confirm that every sponsor can be integrated in time; assign frontend, Neo4j, backend/execution and demo responsibilities; build a functioning end-to-end loop before adding presentation polish.

Closing line: “Most recovery systems ask whether the operation can continue. Exception One asks whether everyone can continue with it.”